

Coding Challenge

[← Back](#)

QUESTIONS

Q1

External Sort Simulation
10 marks

Q2

Resizable Hash Table
10 marks

Q3

Linked List Merge Sort
10 marks

3/3 answered

Q1 External Sort Simulation

10 marks

Simulate external sorting. Input N, chunk size M, then N integers. Sort each chunk, perform a k-way merge, and print the final sorted sequence. On the next line print RUNS r where $r = \text{ceil}(N/M)$.

Your Code

```
1 #include <stdio.h>
2 int main() {
3     int n, m;
4     scanf("%d %d", &n, &m);
5     int a[n];
6     for (int i = 0; i < n; i++)
7         scanf("%d", &a[i]);
8     for (int i = 0; i < n; i += m) {
9         int e = i + m;
10        if (e > n) e = n;
11        for (int j = i; j < e - 1; j++) {
12            for (int k = j + 1; k < e; k++) {
13                if (a[j] > a[k]) {
14                    int t = a[j];
15                    a[j] = a[k];
```

Input

```
10 3
9 4 7 2 8 1 6 3 5 0
```

Output

```
0 1 2 3 4 5 6 0 7 8
RUNS 4
```

Visible Test Cases

Coding Challenge

← Back

QUESTIONS

Q1

External Sort Simulation
10 marks



Q2

Resizable Hash Table
10 marks



Q3

Linked List Merge Sort
10 marks



3/3 answered

Q2 Resizable Hash Table

10 marks

Implement a separate-chaining hash table that doubles when load factor exceeds 0.75. Input initial size S and Q operations I key, S key, D key. Print FOUND/NOT FOUND for searches and finally print SIZE x and COUNT y.

Your Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int key;
6     struct Node *next;
7 };
8
9 struct Node **table;
10 int size;
11 int count = 0;
12
13 void insert(int key) {
14     int index = key % size;
15
16     struct Node *p = table[index];
```

Input

4 6
I 10
I 14
I 20

Output

FOUND
NOT FOUND
SIZE 4
COUNT 2

Visible Test Cases

Test Case 1



Coding Challenge

← Back

QUESTIONS

Q1

External Sort Simulation
10 marks

Q2

Resizable Hash Table
10 marks

Q3

Linked List Merge Sort
10 marks

3/3 answered

Q3 Linked List Merge Sort

10 marks

Read N integers, create a singly linked list, sort it using merge sort with slow/fast splitting, and print the original list followed by the sorted list.

Your Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node* merge(struct Node *a, struct Node *b) {
10     struct Node *r = NULL;
11
12     if (!a) return b;
13     if (!b) return a;
14
15     if (a->data < b->data) {
16         r = a;
17         r->next = merge(a->next, b);
```

Input

6
5 2 8 1 9 3

Output

5 2 8 1 9 3
1 2 3 5 8 9

Visible Test Cases

Test Case 1